1. Position Embeddings Exploration (30 points)

Position embeddings are an important component of the Transformer architecture, allowing the model to differentiate between tokens based on their position in the sequence. In this question, we'll explore the need for positional embeddings in Transformers and how they can be designed.

Recall that the crucial components of the Transformer architecture are the self-attention layer and the feed-forward neural network layer. Given an input tensor $\mathbf{X} \in \mathbb{R}^{T \times d}$, where T is the sequence length and d is the hidden dimension, the self-attention layer computes the following:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V$$
$$\mathbf{H} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V}$$

where $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d \times d}$ are weight matrices, and $\mathbf{H} \in \mathbb{R}^{T \times d}$ is the output.

Next, the feed-forward layer applies the following transformation:

$$\mathbf{Z} = \text{ReLU}(\mathbf{H}\mathbf{W}_1 + \mathbf{1} \cdot \mathbf{b}_1)\mathbf{W}_2 + \mathbf{1} \cdot \mathbf{b}_2$$

where $\mathbf{W}_1, \mathbf{W}_2 \in \mathbb{R}^{d \times d}$ and $\mathbf{b}_1, \mathbf{b}_2 \in \mathbb{R}^{1 \times d}$ are weights and biases; $\mathbf{1} \in \mathbb{R}^{T \times 1}$ is a vector of ones*; and $\mathbf{Z} \in \mathbb{R}^{T \times d}$ is the final output.

(Note that we have omitted some details of the Transformer architecture for simplicity.)

(a) **Permuting the input.**

i. Suppose we permute the input sequence $\mathbf{X}$ such that the tokens are shuffled randomly. This can be represented as multiplication by a permutation matrix $\mathbf{P} \in \mathbb{R}^{T \times T}$, i.e. $\mathbf{X}_{\text{perm}} = \mathbf{P}\mathbf{X}$. (See Wikipedia for a recap on permutation matrices.)
Show that the output $\mathbf{Z}_{\text{perm}}$ for the permuted input $\mathbf{X}_{\text{perm}}$ will be $\mathbf{Z}_{\text{perm}} = \mathbf{P}\mathbf{Z}$.
You are given that for any permutation matrix $\mathbf{P}$ and any matrix $\mathbf{A}$, the following hold: $\text{softmax}(\mathbf{P}\mathbf{A}\mathbf{P}^\top) = \mathbf{P}\ \text{softmax}(\mathbf{A})\ \mathbf{P}^\top$ and $\text{ReLU}(\mathbf{P}\mathbf{A}) = \mathbf{P}\ \text{ReLU}(\mathbf{A})$.

ii. Think about the implications of the result you derived in part i. **Explain** why this property of the Transformer model could be problematic when processing text.

(b) **Position embeddings** are vectors that encode the position of each token in the sequence. They are added to the input word embeddings before feeding them into the Transformer.

One approach is to generate position embedding using a fixed function of the position and the dimension of the embedding. If the input word embeddings are $\mathbf{X} \in \mathbb{R}^{T \times d}$, the position embeddings $\Phi \in \mathbb{R}^{T \times d}$ are generated as follows:

$$\Phi_{(t,2i)} = \sin\left(t/10000^{2i/d}\right)$$
$$\Phi_{(t,2i+1)} = \cos\left(t/10000^{2i/d}\right)$$

where $t \in \{0, 1, \ldots T-1\}$ and $i \in \{0, 1, \ldots d/2-1\}$†.

Specifically, the position embeddings are added to the input word embeddings:

$$\mathbf{X}_{\text{pos}} = \mathbf{X} + \Phi$$

i. Do you think the position embeddings will help the issue you identified in part (a)? If yes, explain how and if not, explain why not.

ii. Can the position embeddings for two different tokens in the input sequence be the same? If yes, provide an example. If not, explain why not.

*Outer product with $\mathbf{1}$ represents broadcasting operation and makes feed forward network notations mathematically sound.
†Here d is assumed even which is typically the case for most models.

2. Attention Exploration (30 points)

Multi-head self-attention is the core modeling component of Transformers. In this question, we'll get some practice working with the self-attention equations, and motivate why multi-headed self-attention can be preferable to single-headed self-attention.

Recall that attention can be viewed as an operation on a *query* vector $q \in \mathbb{R}^d$, a set of *value* vectors $\{v_1, \ldots, v_n\}, v_i \in \mathbb{R}^d$, and a set of *key* vectors $\{k_1, \ldots, k_n\}, k_i \in \mathbb{R}^d$, specified as follows:

$$c = \sum_{i=1}^{n} v_i \alpha_i \tag{1}$$

$$\alpha_i = \frac{\exp(k_i^\top q)}{\sum_{j=1}^{n} \exp(k_j^\top q)} \tag{2}$$

with $alpha = \{\alpha_1, \ldots, \alpha_n\}$ termed the "attention weights". Observe that the output $c \in \mathbb{R}^d$ is an average over the value vectors weighted with respect to α.

(a) **Copying in attention.** One advantage of attention is that it's particularly easy to "copy" a value vector to the output c. In this problem, we'll motivate why this is the case.

i. The distribution α is typically relatively "diffuse"; the probability mass is spread out between many different α_i. However, this is not always the case. **Describe** (in one sentence) under what conditions the categorical distribution α puts almost all of its weight on some α_j, where $j \in \{1, \ldots, n\}$ (i.e. $\alpha_j \gg \sum_{i \neq j} \alpha_i$). What must be true about the query q and/or the keys $\{k_1, \ldots, k_n\}$?

ii. (1 point) Under the conditions you gave in (i), **describe** the output c.

(b) **An average of two.** Instead of focusing on just one vector v_j, a Transformer model might want to incorporate information from *multiple* source vectors.

Consider the case where we instead want to incorporate information from **two** vectors v_a and v_b, with corresponding key vectors k_a and k_b. Assume that (1) all key vectors are orthogonal, so $k_i^\top k_j = 0$ for all $i \neq j$; and (2) all key vectors have norm 1. **Find an expression** for a query vector q such that $c \approx \frac{1}{2}(v_a + v_b)$, and **justify your answer**.[‡] (Recall what you learned in part (a).)

(c) **Simple Calculation:** Given the query vector $q = [3, 3]$ and three key vectors $k_1 = [2, 0]$, $k_2 = [0, 2]$, $k_3 = [2, 2]$, with corresponding value vectors $v_1 = [1, 0]$, $v_2 = [0, 1]$, $v_3 = [0.5, 0.5]$. $k_i^\top q$ denotes the dot product of k_i and q, where $k_i^\top$ is the transpose of k_i to match the dimension for valid dot product calculation.(all are column vectors by default)

i. Compute the dot product $k_i^\top q$ between q and each k_i.

ii. According to the dot product results, determine which α_i has the largest attention weight. You do not need to calculate the exact value; just explain the reasoning.

(d) **Multi-Head Attention Calculation** Now we extend the single-headed attention setup in part (c) to a 2-headed multi-head self-attention mechanism. For each attention head $h \in \{1, 2\}$, we have its own query vector $q_h \in \mathbb{R}^d$, and share the same key vectors $\{k_1, k_2, k_3\}$ and value vectors $\{v_1, v_2, v_3\}$ as in part (c).

The output of head h is computed as:

$$c_h = \sum_{i=1}^{3} v_i \alpha_{h,i}, \quad \alpha_{h,i} = \frac{\exp(k_i^\top q_h)}{\sum_{j=1}^{3} \exp(k_j^\top q_h)}$$

[‡]Hint: while the softmax function will never *exactly* average the two vectors, you can get close by using a large scalar multiple in the expression.

The final multi-head output is the average of the two head outputs:

$$c_{\text{multi}} = \frac{1}{2}(c_1 + c_2)$$

Given the new query vectors:

- Head 1 query: $q_1 = [3, 0]^\top$
- Head 2 query: $q_2 = [0, 3]^\top$

i. For each head $h = 1, 2$, compute the dot product $k_i^\top q_h$ between the query and each key, then determine which $\alpha_{h,i}$ has the largest attention weight for that head (no need to calculate exact values, just explain the reasoning).

ii. Compare the final multi-head output c_{multi} with the single-head output c from part (c). Explain why multi-head attention can capture information from multiple value vectors simultaneously, while the single-head attention in (c) only focuses on a single value vector.

3. Efficient Attention Design for Long-Context Inference (40 points)

A team is deploying a decoder-only LLM for long-context question answering. The model configuration is as follows:

- sequence length $N = 4096$
- number of layers $L = 24$
- number of query heads $h_q = 16$
- head dimension $d = 64$
- in standard multi-head attention, each query head has its own K and V
- autoregressive decoding uses KV Cache
- all cached tensors are stored in FP16, so each element takes **2 bytes**

The team is comparing the following options:

1. standard multi-head attention + KV Cache
2. GQA: 16 query heads are divided into 4 groups, and each group shares one K and one V
3. MQA: all 16 query heads share the same K and the same V
4. FlashAttention: does not change the mathematical result, but optimizes the memory access pattern

Recall that in autoregressive decoding, each layer stores past key and value tensors for all previously generated tokens. In this question, you will analyze the storage cost of different attention variants and explain why some methods mainly improve memory access efficiency rather than directly reducing cache size.

(a) **Standard multi-head attention with KV Cache**

In standard multi-head attention, each query head has its own K and V. Assume that for each layer, both K and V must be cached for all past tokens.

i. For **standard multi-head attention**, compute the total number of cached K elements and the total number of cached V elements in **one layer** during inference.

ii. Based on your result in part (i), compute:

- the total number of cached elements (K+V) in one layer
- the total number of cached elements (K+V) across all 24 layers

(b) **Converting cache size into memory usage**

Now convert the cache size from part (a) into actual memory usage. Recall that all cached tensors are stored in FP16, so each element takes 2 bytes.

i. Compute the total cache size (K+V) for **one layer** in bytes and in MB. Use

$$1 \text{ MB} = 1024^2 \text{ bytes}.$$

ii. Compute the total cache size (K+V) across all 24 layers in bytes and in MB.

(c) **Grouped-Query Attention (GQA)**

Now suppose the model uses **GQA**, where the 16 query heads are divided into 4 groups, and all query heads in the same group share one K and one V.

i. Compute the total number of cached elements (K+V) in **one layer** under GQA.

ii. Compute the total cache size (K+V) across all 24 layers in MB.

iii. Compare your answer with standard multi-head attention. By what factor does the cache shrink?

iv. Briefly explain why GQA can reduce KV cache size without reducing the number of query heads.

(d) **Multi-Query Attention (MQA)**

Now suppose the model uses **MQA**, where all 16 query heads share a single K and a single V.

i. Compute the total number of cached elements (K+V) in **one layer** under MQA.

ii. Compute the total cache size (K+V) across all 24 layers in MB.

iii. Compare the cache sizes of **standard MHA, GQA, and MQA**. State the ordering from largest to smallest, and briefly summarize the trend.

(e) **FlashAttention and KV Cache**

The team is also considering **FlashAttention**. Recall that FlashAttention does not change the mathematical definition of attention, but improves efficiency by changing how attention is computed and stored temporarily.

i. What bottleneck does FlashAttention mainly optimize: arithmetic computation, or memory reads/writes? Briefly explain.

ii. State the two main ideas behind FlashAttention.

iii. Does FlashAttention change the mathematical result of attention? Explain briefly.

iv. Based on the lecture, does FlashAttention directly reduce the size of KV cache? Why or why not?

Lab Course Assignment 2: Transformer Architecture

Covering Labs 4-6: Attention Mechanisms, Transformer Architecture

Submission Instructions

Academic Honesty: This assignment must be completed individually. High-level conceptual discussions are allowed, but sharing code or solutions is strictly prohibited.

Statement on AI tools: You may use LLMs for low-level programming or conceptual queries. However, using them to solve the core assignment is prohibited. We strongly recommend disabling AI autocomplete features (e.g., Cursor Tab) during implementation.

1. **Environment:** Use the provided `pyproject.toml` to set up your environment.
2. **Implementation:** Complete the designated `TODO` blocks in `p1_modern_components.py`, `p2_kv_cache.py`, and `p3_minigpt.py` without modifying existing function signatures.
3. **Submission:** Zip only your implementation files (renamed as `ID_name_assignment2.zip`) and submit. Do not include `.venv` or `__pycache__` directories.

Problem 1: Modern Transformer Components (30 pts) `p1_modern_components.py`

Motivation: In modern Large Language Models (e.g., LLaMA, Qwen), classical components like Layer Normalization and standard MLPs have been superseded by more efficient variants. Implementing these will give you a hands-on understanding of what powers state-of-the-art architectures.

Part A: Root Mean Square Normalization (RMSNorm) (15 pts)

RMSNorm simplifies LayerNorm by removing the mean-centering step, reducing computational overhead while maintaining convergence properties.

Mathematical Formulation: For an input $x \in \mathbb{R}^d$, the output is defined as:

$$\mathrm{RMS}(x) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \varepsilon}$$

$$y = \frac{x}{\mathrm{RMS}(x)} \odot w$$

where $w \in \mathbb{R}^d$ is a learnable scaling parameter. Implement the forward pass in `RMSNorm` ensuring numerical stability using the `eps` parameter.

Part B: Swish Gated Linear Unit (SwiGLU) (15 pts)

SwiGLU replaces the standard MLP by using three linear projections (`w_gate`, `w_up`, `w_down`) and gating the `up` projection with a Swish-activated `gate` projection.

Mathematical Formulation: For $\beta = 1$, Swish is identical to SiLU:

$$\mathrm{Swish}(z) = z \odot \sigma(z)$$

$$\mathrm{SwiGLU}(x) = \big(\mathrm{Swish}\big(xW_{\text{gate}}\big) \odot \big(xW_{\text{up}}\big)\big)W_{\text{down}}$$

Implement this forward pass inside the `SwiGLU` class.

Problem 2: KV Cache Self-Attention (30 pts) `p2_kv_cache.py`

Motivation: During auto-regressive generation, a language model generates one token at a time. A naive implementation of self-attention would recompute the Keys and Values for all past tokens at every step, leading to an $O(N^2)$ decoding complexity. To solve this, generation engines cache the historical Keys (K) and Values (V).

Implementation Task: In `p2_kv_cache.py`, implement `KVCacheAttention` for optimized step-by-step decoding.

1. **Cache Update:** Concatenate the new k and v to `self.k_cache` and `self.v_cache` along the sequence dimension.
2. **Attention:** Compute Scaled Dot-Product Attention using the **current** query q_t and the **full** cached Keys (K_{full}) and Values (V_{full}).

Inference Time Comparison: Once implemented, run `uv run python p2_kv_cache.py` to execute the provided benchmark. This script compares the time taken to generate a sequence of 500 tokens with and without KV-caching. Observe the significant speedup achieved!

> **Implementation Hint:** When concatenating tensor caches, ensure you are operating on the sequence dimension (usually `dim=1`).

Problem 3: MiniGPT Training & Generation (40 pts) `p3_minigpt.py`

Motivation: Bring everything together to build a `MiniGPT` from scratch and train it on the TinyShakespeare dataset to generate text autonomously!

Auto-Regressive Language Modeling: GPT models operate on the principle of next-token prediction. In **Parallel Training**, the model processes a sequence simultaneously using a causal mask to ensure high-efficiency next-token forecasting. In **Sequential Inference**, the model generates text step-by-step by predicting the next token, appending it to the context, and repeating the cycle.

Part A: Architecture Assembly (10 pts)

Implement the `forward` function in `p3_minigpt.py`. Your code should convert `input_ids` to embeddings (scaled by $\sqrt{d_{\text{model}}}$), apply positional encoding, and generate a causal mask. Then, pass the sequence through the transformer blocks and project the hidden states to the vocabulary size using the language model head.

Part B: Causal Language Modeling Loss (10 pts)

To train the model, implement the loss calculation by shifting the logits and target `input_ids` to align them for `CrossEntropyLoss`. Specifically, the logits at position t (i.e., `logits[:, :-1, :]`) should be trained to predict the token at $t+1$ (`input_ids[:, 1:]`).

Part C: Greedy Auto-Regressive Generation (10 pts)

Implement `generate(input_ids, max_new_tokens)` to perform inference. In each iteration, the model performs a forward pass, extracts the logits for the final time step, and uses `torch.argmax` to pick the most likely next token to append to the input sequence.

Part D: Training Loop and TinyShakespeare (10 pts)

Complete the PyTorch training loop in `train_minigpt` using the provided `tinyshakespeare.txt`. You must implement the 4 standard optimization steps: zeroing gradients, computing loss, backpropagation, and updating the optimizer.

Grading Note: Your grade is based strictly on the correctness of your implementation. Due to potential hardware variations among students, we do not require a specific final accuracy or loss value for full credit.

Bonus Exploration: We encourage you to experiment with hyperparameters—such as increasing model depth/width, adjusting learning rates, or use more training data see how high you can push the validation accuracy!

Recommended Compute Resources:

Device Type	Recommended Hardware	Est. Training Time (10 Epochs)
CPU	Intel i5 Core / AMD R5 or better	10–20 Minutes
GPU (NVIDIA)	RTX 3060 / 4060 or better	2–4 Minutes
Apple Silicon	Mac M1 / M2 / M3 (MPS)	8–15 Minutes

— End of Assignment 2 —